

Research Proposal

An External Route-Memory Constraint Architecture for Long-Horizon Vision-Language Navigation

Bojun Long

Student

Supervisor: Chengxu Zhou

Day-to-Day Supervisor: Tianhu Peng

Project Mode: In-Person

20 May 2026

Abstract

With the breakthroughs of vision-language navigation models (VLN, e.g. NaV-ILA), embodied agents have made remarkable progress on short-range physical navigation. However, because such models mostly generate actions autoregressively in a step-by-step fashion, small errors arising early on accumulate continuously along the trajectory and gradually drive the agent away from its intended state—this “drift” phenomenon is particularly pronounced in long-horizon tasks, and existing models generally lack effective mechanisms for online correction and state recovery. Among the various long-horizon tasks, this project focuses on a class of persistent navigation tasks with a *round-trip structure* (Long-Horizon Persistent Navigation): the agent must first travel to a target according to an instruction (outbound), perform a confirmation at the target (confirm), and then return to the starting point (return). A defining characteristic of this setting is that the return phase revisits environmental regions already traversed during the outbound phase, so that the outbound trajectory itself inherently embodies spatial priors that can be reused on the way back. Based on this observation, this project proposes, without altering the overall weights of the VLN backbone, to attach to it an **explicit External Memory Space**: this memory continuously records the experiential information generated during execution in the outbound phase, and is invoked in reverse during the return phase to serve as an external reference for the VLN’s localization, phase estimation, and drift correction.

1 Literature Review

Vision-and-Language Navigation (VLN) requires an embodied agent to complete navigation tasks in unknown environments according to natural language instructions. Since the introduction of Room-to-Room (R2R) [1], VLN has become one of the core research directions in embodied AI. Early work relied mainly on the real

indoor scenes provided by Matterport3D [2] and interactive simulation environments such as AI2-THOR [3], evaluating single-instruction following on datasets such as R2R [1] and RxR [4]. Subsequently, with the introduction of a continuous action space by VLN-CE [5], the task setting moved further toward realistic robot deployment scenarios. The recent survey *Vision-and-Language Navigation Today and Tomorrow* [6] systematically summarizes the development of VLN in the era of foundation models, and notes that the field has evolved from an early mapping problem centered on vision-language alignment into a broader research direction that must simultaneously coordinate world representation, user-intent understanding, and embodied decision-making.

At the methodological level, mainstream research can be broadly divided into two complementary paradigms. The first directly leverages vision-language foundation models to drive action decisions: NaVILA [7] trains a VLM into a VLN-oriented VLA that outputs mid-level phrase-style navigation commands; DualVLN [8] decomposes navigation into a System 2 responsible for pixel-goal selection and a System 1 responsible for trajectory generation; and Uni-NaVid [9] unifies VLN, ObjectNav, Embodied QA, and other tasks into a single video-language-action framework. The second paradigm introduces waypoints, candidate nodes, or topological graphs as explicit intermediate representations: Skill-Nav [10] couples a classical planner with an LLM planner on top of strong locomotion capabilities; NavGPT-2 [11] aligns multi-view observations into the hidden space of an LLM for candidate-node scoring; DUET [12] jointly models global topology and local observations to strike a balance between long-horizon planning and local decision-making; MapGPT [13] organizes the explored environment into a discrete structured graph suited to LLM comprehension; SmartWay [14] uses an occupancy map to filter geometric traversability before waypoint generation; and ETPNav [15] self-organizes online-generated waypoints into a topological graph as a structured resource for long-horizon planning. These two paradigms emphasize different aspects—the former stresses end-to-end semantic reasoning, while the latter stresses the interpretability and controllability of spatial structure—but both were initially built upon the task assumption of “a single-shot instruction, a single goal, and a short horizon.”

As VLN research moves toward more realistic deployment scenarios, this short-horizon assumption is rapidly being challenged. *Towards Long-Horizon Vision-Language Navigation* (LH-VLN) [16] formally poses long-horizon VLN as an independent research problem, pointing out that existing VLN methods struggle with long-horizon tasks not merely because the instructions are longer, but also because the required memory length grows substantially, the context burden increases, and stage-wise success becomes harder to assess. Its experiments further show that existing models—including ETPNav [15], NaviLLM, and GPT-4 + NaviLLM—all achieve a zero success rate on long-horizon tasks comprising two to three sub-tasks. *Navigating the Nuances* [17], through a fine-grained evaluation framework, further reveals systematic shortcomings of these models along capability dimensions such as value understanding, phase-order judgment, and region recognition in long-horizon tasks. Together, these two works transform long-horizon VLN from a vague, intuition-driven problem into a research object that can be systematically

compared and diagnosed, and make “drift”—the phenomenon whereby a model’s state, spatial representation, and task semantics gradually deviate from expectation during execution—a central analytical lens for this direction.

Around the drift problem in long-horizon VLN, existing research has formed several distinct lines of discussion; among them, the more representative works can be broadly grouped into two interrelated directions. The first line concerns the online rectification of state-distribution drift: once a model deviates at some step from the expert distribution seen during training, subsequent observations gradually depart from the training distribution, and the error keeps amplifying over time [18]. DAgger [19] established the classical paradigm of this direction: it continually collects expert corrections on the state distribution induced by the policy itself and folds these corrections back into training, thereby expanding the model’s coverage of erroneous states. CorrectNav [20] further treats erroneous trajectories themselves as high-value training data, automatically locating the onset of deviation and specifically learning recovery trajectories that reconnect to the correct path. BudVLN [21] points out that geometrically forcing the agent back onto the original trajectory may conflict with the semantics of the original instruction; it therefore proposes returning to “the last position that is still semantically correct” and restarting from there, thereby shifting the rectification objective from geometric consistency to semantic consistency. Visual teach-and-repeat (VT&R) [22] represents another line of thought: the system first records a reliable route template during a teaching phase, then continually performs visual drift correction during the repeat phase, exhibiting stable robustness to strong disturbances along the taught route. Overall, this line characterizes the evolution of rectification mechanisms—from passively “returning to the expert trajectory” toward actively “imposing online constraints based on prior route experience.”

The second line concerns the explicit management of subtask progress and phase-boundary drift. In long-instruction tasks, the system must not only execute actions but also judge which phase of the task it is currently in, whether prior subtasks have been completed, and when it should switch to the next phase. Early VLN models typically compressed such judgments implicitly into an LSTM hidden state or Transformer attention, and thus lacked both interpretability and reliability. SeqWalker [23] decomposes a long instruction into multiple phrases, uses CLIP’s text-visual similarity to determine which fragment of the instruction the current observation corresponds to, and triggers a rollback when a phase-order inconsistency is detected. By contrast, Open-Nav [24] uses a local open-source LLM to decompose a long instruction into decomposed actions and extracted landmarks; during execution it explicitly tracks which landmarks have been observed and which actions have been completed, and introduces an ordering constraint whereby “subsequent actions must be predicated on the completion of preceding ones.” These two works convert the progress-tracking process—formerly hidden inside the model—into explicit process variables that can be reasoned about and constrained, marking a shift in phase-boundary management from an implicit hidden state toward explicit progress estimation.

Finally, a line of research directly relevant to this project is the introduction

of explicit external memory into VLN. In cross-instruction continuous navigation settings (e.g. IVLN/IVLN-CE), existing work has found that agents maintaining persistent, spatially structured external memory significantly outperform those relying solely on unstructured implicit memory, thereby advocating architectural designs based on explicit spatial representations [6]. Along this line, Mem2Ego [26] adaptively aligns global memory with first-person local observations to enhance spatial reasoning in long-horizon tasks; the Multi-Granularity Dynamic Memory proposed in LH-VLN [16] coordinates long- and short-term memory to alleviate memory overload in long-horizon tasks; and MSNav [27] points out that end-to-end decision-making by a single large model in long-horizon tasks suffers from weak spatial reasoning, insufficient cross-modal grounding, and memory overload, which it mitigates with a dedicated memory module. Taken together, this body of work indicates that externalizing the persistent memory required for long-horizon navigation out of the model’s internal implicit state, and entrusting it to an explicit module, is an effective route to alleviating long-horizon drift.

2 Research Gaps

It can thus be seen that, although VLN has successfully mapped language instructions into a continuous physical action space and substantially improved the precision of low-level action alignment, long-horizon tasks still entail core problems such as state drift and subtask progress drift. At the same time, although existing research has shown that explicit external memory helps alleviate long-horizon drift, existing external memories are mostly designed for “arbitrary goal queries” in single-trip tasks, and relatively little attention has been paid to how, in long-horizon tasks with a round-trip structure, already-executed experience can be organized into a *reusable* external memory, and how the environmental and trajectory information recurring between the outbound and return phases can be exploited to support localization, progress maintenance, and drift correction during the return phase. Moreover, during the return phase the robot often re-observes the same environmental regions from a viewpoint nearly opposite to that of the outbound phase, producing a pronounced viewpoint-distribution discrepancy between current observations and historical memory, which further increases the difficulty of matching and state estimation based on historical memory.

This research is therefore built upon the following three gaps:

Gap A: Existing work handles state-distribution drift insufficiently to support long-horizon execution. Imitation-learning-based VLN agents are fundamentally subject to state-distribution drift: small step-wise deviations accumulate quadratically along the trajectory, pushing the agent into states never seen during training [19, 18]. This problem is especially severe in long-horizon VLN, because expert demonstration trajectories contain almost no behavior for recovering from out-of-distribution states [25]. Although some work has begun to address heading correction and online rectification during navigation, most existing methods focus

on local recovery—that is, pulling the policy back onto a feasible trajectory only after deviation has already occurred. While such ideas can alleviate local errors in short-range navigation, they still lack a sustained suppression mechanism oriented toward the long-horizon execution process, and thus cannot prevent small errors from continually accumulating across multi-stage tasks and ultimately evolving into overall failure.

Gap B: Existing work handles subtask progress and phase-boundary drift insufficiently robustly. Although recent methods have begun to explicitly model task progress and phase switching, their decision processes generally still rely heavily on semantic cues such as language decomposition, landmark recognition, or vision-language matching. This design makes phase judgments highly susceptible to a cascade of upstream decomposition errors, perception errors, and historical-summary errors, leading to premature, delayed, or out-of-order subtask switching in long-horizon tasks. Therefore, while existing methods have improved the interpretability of progress modeling, they still lack a constraint mechanism stable enough to independently support phase determination during execution.

Gap C: Existing work lacks a reusable external-memory representation for long-horizon round-trip tasks. Although recent VLN systems have begun to introduce explicit maps, topological graphs, or long-term spatial memory to assist navigation, and have demonstrated the value of external memory in alleviating long-horizon drift, these memory structures mostly still serve current decision-making and local planning in single-trip tasks; their organization targets the general need of “querying an arbitrary goal from an arbitrary position,” and they have not yet been organized into a form capable of supporting *reuse in round-trip tasks*. In particular, in long-horizon persistent VLN the robot actually re-traverses already-visited environmental regions during the return phase, so the outbound experience itself naturally contains reusable prior information. Yet existing methods generally lack a mechanism that can explicitly record, index, and *reverse*-invoke outbound memory, and seldom study how to exploit the information recurring between outbound and return to support localization, progress maintenance, and online correction during the return phase. Consequently, the VLN field still lacks a reusable, reversible, and structurally organized external-memory representation oriented toward long-horizon round-trip tasks.

In summary, although existing VLN research has begun to touch upon the key difficulties of long-horizon execution and has recognized the value of external memory, it still lacks an external-memory constraint mechanism that is explicitly designed for the round-trip structure, reversible, and robust to viewpoint reversal, all without compromising the VLA’s semantic priors.

3 Core Research Question

Based on the gaps above, the research question we pose is: how can a *reusable and reversible* external memory space be attached to a VLN model, such that it records, during the outbound phase, experiential information that can be reused on the return, and provides the VLN with the explicit spatial reference and correction capability required for round-trip (Return) tasks, while *not* depriving the underlying VLN of its semantic decision-making autonomy.

4 Proposed Method

This project abandons the execution paradigm of traditional end-to-end VLN, which relies entirely on implicit historical memory, and instead proposes to attach an explicit external memory space to the VLN. The following subsections answer three questions in turn: what exactly is stored in this memory space (§4.1 and §4.2), in what manner this information is fed back into the VLN (§4.4 and §4.5), and how the system falls back when the VLN itself becomes incapacitated (§4.6).

4.1 The Form of External Memory: Why Store a Route Rather than a Map

External memory can in principle take various forms—a dense map, a semantic topological graph, or a route template. This project chooses to organize the outbound experience into a **reversible route template**, because in long-horizon round-trip tasks the robot re-traverses, almost segment by segment, the very trajectory it walked during the outbound phase; the outbound trajectory therefore naturally constitutes the optimal reference for the return phase, without any need to build and maintain a complete environment map for the general need of “querying an arbitrary goal from an arbitrary position”—which, for a round-trip task, is both redundant and incurs additional construction and maintenance overhead. This idea is in the same spirit as Visual Teach-and-Repeat [22]: a reliable route is recorded during the teaching phase, and the robot executes along and continually corrects against that route during the repeat phase. This project can be regarded as transferring this classical paradigm to the long-horizon round-trip setting driven by VLN semantic instructions.

4.2 Template Construction and Inversion

This project abstracts the above external memory into a structured template of “topological nodes + local descriptors”:

- **Outbound construction:** The VLN leads exploration according to semantic instructions, while the Agent drops topological anchors in the background at sparse step intervals. Each anchor stores absolute coordinates, accompanied by the current local height map (descriptors) and a local LiDAR point-cloud map as additional verification information.

- **Return inversion:** After the robot turns around, the Agent reverses the template sequence and inverts the relative geometric relations (the Reversed Return Template), which serves as the absolute reference frame for return guidance.

4.3 Dynamic Instruction Decomposition and Task-Phase Management

To address the context overload caused by long instructions, the system introduces a high-level large language model (LLM) together with a state machine for task lifecycle management:

- **Offline decomposition:** The LLM first decomposes the long-horizon round-trip instruction into three subtasks—Outbound, Confirm, and Return—and extracts the corresponding semantic landmarks.
- **Online determination:** At runtime, the Agent maintains a real-time belief about the current task: at each topological anchor, it converts the depth data acquired by the camera into a local point cloud and matches it against every local point cloud stored at the historical anchors. A successful forward match indicates that the robot is backtracking—the current phase is outbound; a successful reverse match proves that the robot has previously visited this anchor—the current phase is return; a failed match indicates that the robot is reaching this anchor for the first time—the current phase is outbound.

Limitation: If the robot encounters an impassable path and must turn around, the method may yield a successful reverse match and thus incorrectly judge the subtask progress to be return. For now, we therefore assume that the experimental scenario contains no impassable paths.

- **Direct Confirm phase:** During the Confirm phase, the system bypasses the VLN model’s autoregressive generation, and a deterministic script controls the chassis to execute preset actions (e.g. a 360° scan to confirm the target), after which it smoothly transitions into the Return phase.

4.4 The Mode of Information Feedback

The approach taken by this project is to inject into the VLN a low-dimensional geometric deviation—namely the translational deviation Δt and the heading deviation $\Delta \theta$ of the current observation relative to the nearest template anchor. This deviation is computed instantaneously by the Agent at each anchor via *local* registration (between the current local observation and the descriptor archived at that anchor), and is then injected into the prompt as structured text (e.g. [System Hint: drifting right by $\Delta \theta^\circ$, next anchor 1.5m ahead]) as one basis for the VLN’s autonomous decision-making. Injecting such structured geometric cues into the prompt already has a successful precedent: NavGPT-2 [11] injects directional information into the navigation prompt in a structured format.

Alternative Route One: feeding the constructed map directly as VLM input. This route suffers from two problems. The first is *redundancy*: as discussed in §4.1, the goal of the return in a round-trip task is to retrace the outbound path, and the outbound trajectory is already a sufficient and optimal reference for that goal; maintaining an additional complete environment map that serves “arbitrary goal queries from arbitrary positions” is a cost paid for a general capability the round-trip task does not need. The second is *global accumulated error*: an environment map is the product of registering and stitching local observations from successive frames into a single global representation, and its registration error keeps accumulating and drifting along the trajectory, so that the farther a region is from the start, the larger its discrepancy from the real environment typically becomes. Should the VLN make judgments based on such a map whose global consistency has already degraded, it would inherit the accumulated error therein.

Alternative Route Two: transcribing the map into text before feeding it to the VLN. Beyond inheriting the above redundancy and global accumulated error, this route additionally introduces a representation-conversion loss. Existing research has repeatedly pointed out that compressing high-dimensional spatial structures such as a *complete scene or map* into limited text inevitably entails significant information loss: TopV-Nav [28] notes that map-to-text conversion loses key spatial information such as room layout, and TagaVLM [29] likewise notes that the two-stage vision-to-text pipeline struggles to fully preserve and digest fine-grained visual information.

How this project’s approach simultaneously avoids the above problems. Regarding the information loss faced by “map-to-text,” it should be noted that its root cause lies in the object being transcribed being a high-dimensional map or scene representation, whose high-dimensional spatial structure can hardly be fully carried by limited text. The information injected by this project is not a map, but a two- to three-dimensional scalar deviation ($\Delta t, \Delta \theta$); for such a low-dimensional geometric quantity, textual and numerical expression is nearly lossless, and there is no issue of spatial-layout information being lost in compression. Moreover, this project does not assume that the route template or the anchors themselves are free of error—both are likewise subject to sensing and registration noise; the distinction lies in the differing strength of the conditions on which the two representations depend. A map representation depends on the strong condition of global consistency, which degrades with distance due to the registration drift produced by successive stitching, and any judgment relying on it inherits the accumulated error accordingly. By contrast, this project obtains only a single local, relative, instantaneous registration result at each anchor, depending on the markedly weaker condition of “per-anchor local registrability,” and is therefore not subject to global accumulated drift. In sum, injecting a low-dimensional deviation as text both avoids the redundancy and global accumulated error inherent in feeding a map directly, and avoids the high-dimensional compression loss inherent in map-to-text conversion, while allowing the geometric constraint to be digested directly through the VLN’s

language-reasoning channel, thereby preserving its pre-trained semantic priors without modifying the model weights.

4.5 System Execution Flow and Prompt-Hinting Pipeline

During the Return Phase, the external memory provides constraints to the VLN through the following flow, while the VLN *always remains the decision-making subject*:

1. **Topological re-localization:** At each step the robot moves, the Agent receives the current local LiDAR map M_{current} and registers it against the descriptor M_{anchor} of the template anchor, computing the translational deviation Δt and the heading deviation $\Delta \theta$.
2. **Text-based prior pre-fusion (Prompt Hinting):** Before the VLN performs inference on the RGB image, the Agent converts the geometric deviation into a structured prompt (e.g. [System Hint: You are drifting right by $\Delta \theta$ degrees, next anchor is 1.5m ahead]) and dynamically injects it into the current instruction. This endows the VLN with progress awareness and allows it to reason about correction *autonomously* in combination with visual semantics, outputting the final action A_{vla} .

Under normal execution, the external memory participates in decision-making only in the form of textual cues, and the VLN always retains the autonomy to adopt, partially adopt, or ignore that cue according to its own visual-semantic understanding. For instance, when the VLN actively deviates from the template trajectory in order to avoid a dynamic obstacle, such deviation should be regarded as a legitimate autonomous decision, and the system will not forcibly intervene merely because the current position has departed from the template.

4.6 Fallback Arbitration

Although the VLN retains full autonomy under normal conditions, in practice it may still fall into a state of *self-incapacitation*. For this purpose, the system establishes a fallback safety net with explicit trigger conditions. Specifically, the VLN is judged to be incapacitated if either of the following two complementary branches holds:

- **Branch One (behavioral level):** Within a window of N consecutive steps, the robot’s net displacement falls below a threshold ε (it has walked yet effectively gone nowhere), or its action sequence exhibits back-and-forth mutual cancellation (e.g. alternating left-turn-right-turn-left-turn). This branch captures the case in which “the VLN makes no progress on the task at the action level” (N is a hyperparameter to be determined, taken as 3-5 in experiments).
- **Branch Two (output level):** The VLN’s language output cannot be parsed into a valid navigation action command. On the baseline we adopt, NaVILA [7], we observe that when the input frames are insufficient to support a judgment,

the model does not output a clear next-step command but instead degenerates into a meaningless utterance; the downstream module, receiving no valid command, then defaults to moving forward. This case of “still moving forward, but with the decision already invalid” cannot be captured by Branch One, and therefore an *instruction-parsability check* is added after the VLN output: if the output can be mapped to a legal action primitive (forward / turn / stop, etc.) it is passed through, and if parsing fails or the output falls into a meaningless category, the decision is judged to have failed.

Both branches examine *only the VLN’s own state* (whether the action makes progress, whether the output can be parsed) and are unrelated to “how far it is from the template,” and thus do not constitute a deprivation of the VLN’s autonomy. Once either branch is triggered, the system performs a **brief escape**: the action is temporarily switched to the geometry-based correction suggestion $A_{\text{agent}}(\Delta t, \Delta \theta)$ generated by the Agent, advancing one or a few steps toward the nearest successfully matched anchor to break the deadlock, and control is then *immediately handed back to the VLN*, rather than taking over the entire return.

5 Deployment and Fine-Tuning Strategy

Through explicit external route memory, geometric re-localization, and the fallback mechanism, this project provides external constraints to the VLN at inference time, allowing it to extend to long-horizon round-trip tasks while retaining its original semantic navigation capability. However, the effectiveness of the external constraints depends on two capabilities that cannot be guaranteed by inference alone: first, the VLN must be able to correctly understand and respond to the geometric-deviation hints injected as text; and second, the VLN must be able to execute stably in the target physical environment. To this end, while keeping the backbone weights entirely frozen, this project introduces two parameter-efficient fine-tuning (LoRA) procedures, so as to avoid both the high cost of full-parameter training and catastrophic forgetting of the pre-trained semantic priors.

5.1 Hint-Response Alignment

A prerequisite for injecting geometric deviations as structured text is that the VLN can correctly map such hints to corresponding correction actions. However, prompt formats of this kind (e.g. [System Hint: drifting right by $\Delta \theta^\circ$]) lie outside the VLN’s pre-training distribution, and the model may not necessarily make the expected action adjustments accordingly. To bridge this gap, this project automatically synthesizes “hint-action” alignment data in a simulation environment (VLN-CE / Isaac Sim): known perturbations are applied to expert trajectories, the true translational and heading deviations ($\Delta t, \Delta \theta$) are computed directly from the perturbations, the corresponding structured hint text is generated accordingly, and the “action required to return to the expert trajectory” is used as

the supervision label. This pipeline can batch-generate training samples with deviation hints without manual annotation, enabling the VLN, via LoRA fine-tuning, to acquire the ability to “read a geometric hint and output the corresponding correction action,” while preserving its original semantic navigation priors thanks to the frozen backbone.

5.2 Target-Environment Adaptation

The VLN’s pre-trained weights derive mainly from public indoor datasets such as Matterport3D, and their visual domain and action responses may not transfer directly to this project’s physical deployment environment. Therefore, after deployment on the physical platform (Go2), this project performs lightweight incremental LoRA fine-tuning on the model by collecting a small number of real laboratory trajectories, so as to fit the laboratory’s specific differences in lighting, layout, and dynamics.

5.3 Deployment Architecture

In terms of system implementation, the robot side is responsible only for sensor-data acquisition and low-level locomotion execution, while the high-level VLN inference and template-arbitration logic run on an external compute device. The system connects to the low-level controller through a standardized velocity interface (e.g. `cmd_vel`), and can therefore be adapted relatively easily to existing quadrupedal or humanoid robot platforms.

6 Expected Contributions

1. **Mechanism innovation:** We propose a *reusable external route-memory* mechanism oriented toward long-horizon round-trip tasks, which can organize the outbound experience into a reversible, reusable structured prior, and provide the VLN system with online localization, phase maintenance, and drift correction during the return phase.
2. **An autonomy-preserving constraint paradigm:** We establish a two-tier constraint logic of “pre-fusion textual hints (normal) + fallback escape upon incapacitation (exceptional),” which provides the VLN with an external spatial reference while preserving its semantic decision-making autonomy, thereby being mechanistically distinct from hard-coded route following.
3. **Engineering value:** We provide a deployment scheme based on real hardware (RealSense + Unitree) in which the backbone model is entirely frozen and only parameter-efficient fine-tuning (LoRA) is used to accomplish hint-response alignment and target-environment adaptation, substantially lowering the research and deployment cost of long-horizon VLN algorithms while preserving the pre-trained semantic priors.

7 Deliverables Timeline

1. **Experimental environment preparation (May 2026):** Complete the construction of the VLN-CE simulation environment and apply for access to the Matterport3D dataset, establishing the basic runtime environment for subsequent experiments.
2. **Baseline deployment and initial runs (May-June 2026):** Deploy, configure, and preliminarily run the baseline model, establishing the basic navigation framework required for subsequent comparative experiments.
3. **Long-horizon round-trip benchmark construction (May-June 2026):** Construct a three-phase outbound-confirm-return continuous round-trip benchmark within a single episode, used to simulate long-horizon persistent navigation tasks.
4. **Outbound route-template construction (June-July 2026):** Implement the outbound route-template construction module, including sparse anchor recording, local geometric-descriptor storage, and the structured external-memory representation.
5. **Implementation of the return-phase template-reuse mechanism (June-July 2026):** Implement the return-phase Template-Reuse Agent, including reverse-template construction, anchor matching, segment-level progress tracking, and the geometric-correction mechanism.
6. **Integration of the Template-Reuse Agent with the VLA/VLN (July-August 2026):** Integrate the Template-Reuse Agent with the VLA/VLN baseline through prompt-level textual hints and the fallback-escape logic, realizing online correction at the inference stage.
7. **Hint-response alignment data synthesis and fine-tuning (July-August 2026):** Synthesize “hint-action” alignment data in VLN-CE / Isaac Sim, and fine-tune the VLN via LoRA so that it acquires the ability to respond to geometric-deviation hints.
8. **Long-horizon benchmark evaluation (August-September 2026):** Evaluate system performance on the proposed long-horizon benchmark, and compare the navigation performance of the baseline model with and without the external-memory mechanism.
9. **Physical deployment, environment adaptation, and testing (August-September 2026):** Complete the sim-to-real deployment, collect a small number of real laboratory trajectories for incremental LoRA fine-tuning, and conduct physical testing on robot platforms (Go2 / Booster K1 / Unitree G1).

8 Project Logistics

Project Mode. This project is conducted in person. Simulation experiments and model fine-tuning are carried out on on-campus compute resources, and physical deployment and testing are conducted in an on-campus laboratory equipped with Unitree robot platforms; regular communication between the student and the supervisors is conducted online.

Supervision Meetings. This project holds online supervision meetings at a fixed frequency of once every two weeks, conducted via Microsoft Teams, in which the student reports current progress to the supervisor, Chengxu Zhou, and the day-to-day supervisor, Tianhu Peng, discusses technical details, and determines the priorities for the next stage of work. Beyond the regular meetings, the student also stays in touch with the supervisors on specific issues through asynchronous means such as email.

Datasets. The data required for this project fall into two categories: public datasets and self-built data. The public datasets include Matterport3D, which provides RGB-D 3D-scanned scenes of real indoor environments, and VLN-CE, which provides a vision-language navigation benchmark in a continuous action space on top of Matterport3D scenes. This project builds the corresponding VLN-CE simulation environment in Isaac Sim as the main platform for simulation experiments and data synthesis. The above public datasets are all standard resources in the vision-language navigation field and are used under their respective license agreements. The self-built data include the three-phase outbound-confirm-return long-horizon round-trip benchmark constructed within a single episode, and the physical navigation trajectories collected at a small scale in the laboratory for target-environment adaptation.

Ethics Considerations. This project does not involve human subjects, the collection of personal data, or animal experiments: the public datasets used are all standard datasets widely used in the academic community, and the self-collected data consist solely of navigation trajectories recorded by the researcher operating the robot in a controlled laboratory environment, not involving any third party or their identifiable information. As for the physical experiments, given that legged robots possess autonomous locomotion capabilities, the experiments will be conducted in a controlled laboratory environment, with safety measures such as speed limiting, emergency stop, and personnel isolation in place to ensure the safety of personnel and equipment.

References

- [1] P. Anderson et al., “Vision-and-Language Navigation: Interpreting Visually-Grounded Navigation Instructions in Real Environments,” in *Proc. IEEE/CVF*

- CVPR, 2018, pp. 3674–3683.
- [2] A. Chang et al., “Matterport3D: Learning from RGB-D Data in Indoor Environments,” in *Proc. Int. Conf. 3D Vis. (3DV)*, 2017, pp. 667–676.
 - [3] E. Kolve et al., “AI2-THOR: An Interactive 3D Environment for Visual AI,” *arXiv preprint arXiv:1712.05474*, 2017.
 - [4] A. Ku, P. Anderson, R. Patel, E. Ie, and J. Baldridge, “Room-Across-Room: Multilingual Vision-and-Language Navigation with Dense Spatiotemporal Grounding,” in *Proc. EMNLP*, 2020, pp. 4392–4412.
 - [5] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, “Beyond the Nav-Graph: Vision-and-Language Navigation in Continuous Environments,” in *Proc. ECCV*, 2020, pp. 104–120.
 - [6] Y. Zhang et al., “Vision-and-Language Navigation Today and Tomorrow: A Survey in the Era of Foundation Models,” *arXiv preprint arXiv:2407.07035*, 2024.
 - [7] A.-C. Cheng et al., “NaVILA: Legged Robot Vision-Language-Action Model for Navigation,” in *Proc. RSS*, 2025.
 - [8] M. Wei et al., “Ground Slow, Move Fast: A Dual-System Foundation Model for Generalizable Vision-and-Language Navigation,” *arXiv preprint arXiv:2512.08186*, 2025.
 - [9] J. Zhang et al., “Uni-NaVid: A Video-based Vision-Language-Action Model for Unifying Embodied Navigation Tasks,” in *Proc. RSS*, 2025.
 - [10] D. Wang et al., “Skill-Nav: Enhanced Navigation with Versatile Quadrupedal Locomotion via Waypoint Interface,” *arXiv preprint arXiv:2506.21853*, 2025.
 - [11] G. Zhou, Y. Hong, Z. Wang, X. E. Wang, and Q. Wu, “NavGPT-2: Unleashing Navigational Reasoning Capability for Large Vision-Language Models,” in *Proc. ECCV*, 2024.
 - [12] S. Chen, P.-L. Guhur, M. Tapaswi, C. Schmid, and I. Laptev, “Think Global, Act Local: Dual-Scale Graph Transformer for Vision-and-Language Navigation,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 16537–16547.
 - [13] J. Chen, B. Lin, R. Xu, Z. Chai, X. Liang, and K.-Y. K. Wong, “MapGPT: Map-Guided Prompting with Adaptive Path Planning for Vision-and-Language Navigation,” in *Proc. ACL*, 2024.
 - [14] X. Shi et al., “SmartWay: Enhanced Waypoint Prediction and Backtracking for Zero-Shot Vision-and-Language Navigation,” *arXiv preprint arXiv:2503.10069*, 2025.
 - [15] D. An et al., “ETPNav: Evolving Topological Planning for Vision-Language Navigation in Continuous Environments,” *IEEE TPAMI*, vol. 46, no. 8, pp. 5130–5145, 2024.

- [16] X. Song et al., “Towards Long-Horizon Vision-Language Navigation: Platform, Benchmark and Method,” in *Proc. IEEE/CVF CVPR*, 2025.
- [17] Z. Wang, J. Li, Y. Hong, S. Wang, Q. Wu, and S. Gould, “Navigating the Nuances: A Fine-Grained Evaluation of Vision-Language Navigation,” in *Findings of EMNLP*, 2024.
- [18] R. Liu and W. Wang, “Vision-Language Navigation with Energy-Based Policy,” in *Proc. NeurIPS*, 2024.
- [19] S. Ross, G. Gordon, and D. Bagnell, “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning,” in *Proc. AISTATS*, 2011, pp. 627–635.
- [20] Z. Yu et al., “CorrectNav: Self-Correction Flywheel Empowers Vision-Language-Action Navigation Model,” *arXiv preprint arXiv:2508.10416*, 2025.
- [21] G. He et al., “Nipping the Drift in the Bud: Retrospective Rectification for Robust Vision-Language Navigation,” *arXiv preprint arXiv:2602.06356*, 2026.
- [22] P. Furgale and T. D. Barfoot, “Visual Teach and Repeat for Long-Range Rover Autonomy,” *J. Field Robot.*, vol. 27, no. 5, pp. 534–560, 2010.
- [23] Z. Han et al., “SeqWalker: Sequential-Horizon Vision-and-Language Navigation with Hierarchical Planning,” *arXiv preprint arXiv:2601.04699*, 2026.
- [24] Y. Qiao et al., “Open-Nav: Exploring Zero-Shot Vision-and-Language Navigation in Continuous Environment with Open-Source LLMs,” in *Proc. IEEE ICRA*, 2025.
- [25] H. Tian et al., “Dagger Diffusion Navigation: DAgger Boosted Diffusion Policy for Vision-Language Navigation,” *arXiv preprint arXiv:2508.09444*, 2025.
- [26] L. Zhang et al., “Mem2Ego: Empowering Vision-Language Models with Global-to-Ego Memory for Long-Horizon Embodied Navigation,” *arXiv preprint arXiv:2502.14254*, 2025.
- [27] “MSNav: Zero-Shot Vision-and-Language Navigation with Dynamic Memory and LLM Spatial Reasoning,” *arXiv preprint arXiv:2508.16654*, 2025.
- [28] “TopV-Nav: Unlocking the Top-View Spatial Reasoning Potential of MLLM for Zero-shot Object Navigation,” *arXiv preprint arXiv:2411.16425*, 2024.
- [29] “TagaVLM: Topology-Aware Global Action Reasoning for Vision-Language Navigation,” *arXiv preprint arXiv:2603.02972*, 2026.